TECHNICAL DOCUMENTATION

Campus Media Engagement System

A full-stack web application for managing campus media events, student participation, faculty supervision, and attendance tracking across academic blocks.

STACK	VERSION	USER ROLES
MERN (MongoDB, Express, React, Node)	1.0.0 — Production Ready	Head Admin · Faculty · Student

TOTAL PAGES	API ENDPOINTS	TEST COVERAGE
31 (15 Head + 8 Faculty + 8 Student)	40+ REST endpoints	75 tests (44 backend + 31 frontend)

1. Purpose & Overview

The Campus Media Engagement System (CMES) is an operational management platform designed for colleges to coordinate campus media activities. It handles student participation, faculty supervision, event attendance, and reporting across multiple academic blocks.

CMES is **not** a social network or a content platform. It is a structured administrative system where every action — from creating an event to marking attendance — is tracked, role-restricted, and auditable.

> The system manages four academic blocks by default: **ET (Engineering & Technology), MT (Management & Technology), FD (Fashion & Design),** and **Pharmacy**. The Head Administrator can add more blocks at any time.

2. Technology Stack & Why Each Was Chosen

Frontend

Technology	Version	Why It Was Used
React	18.3.1	Component-based UI with hooks — ideal for complex role-based dashboards with shared state. Version 18 chosen over 19 for Recharts compatibility.
Vite	6.3.5	Significantly faster dev server and build times compared to Create React App. Native ESM, instant HMR.
React Router	6.30.0	Client-side routing with nested protected routes per role. v6 API used (not v7 — breaking changes).

Technology	Version	Why It Was Used
Tailwind CSS	3.4.19	Utility-first CSS allows rapid consistent styling without leaving JSX. Custom design tokens (moss, terracotta, sand) defined in config. v3 used — v4 has an entirely different setup incompatible with tailwind.config.js.
Axios	1.9.0	Centralized HTTP client with interceptors. The 401 interceptor auto-redirects to login on session expiry without repeating logic in every component.
Recharts	2.15.3	React-native charting library used for the block-wise participation bar chart on the Head dashboard. Requires React 18 — reason for not upgrading to React 19.
Lucide React	0.511.0	Clean, consistent icon set that matches the organic design identity. Tree-shakeable — only used icons are bundled.

Backend

Technology	Version	Why It Was Used
Node.js	18+	JavaScript runtime enabling a single language across frontend and backend, sharing validation logic and data shapes.
Express	5.1.0	Minimal, flexible HTTP framework. v5 used for async error handling improvements — errors thrown in async route handlers are automatically forwarded to the error middleware without try/catch wrappers.
MongoDB	6+	Document database chosen for its flexible schema — events can have varying participant arrays, attendance correction histories, and notification payloads without complex joins. Atlas M0 is free for deployment.
Mongoose	8.16.4	ODM for MongoDB providing schema validation, pre-save hooks (password hashing), virtual fields, and population. The pre-save hook ensures passwords are always hashed — impossible to accidentally store plaintext.
JWT (jsonwebtoken)	9.0.3	Stateless authentication. Tokens are stored in httpOnly cookies — inaccessible to JavaScript, preventing XSS token theft. 24-hour expiry balances security and usability.

Technology	Version	Why It Was Used
bcryptjs	3.0.3	Password hashing with salt rounds of 12. bcryptjs is pure JavaScript — no native bindings required, making it reliable across deployment environments.
Zod	3.25.49	Schema-first validation library. Every API route that accepts a body has a Zod schema — invalid requests are rejected before reaching the controller. Prevents mass assignment and malformed data.
Helmet	8.0.0	Sets 14 security-related HTTP headers automatically (CSP, X-Frame-Options, HSTS, etc.) with a single middleware call.
express-rate-limit	7.5.0	Prevents brute-force attacks. Auth routes are limited to 5 requests per 15 minutes in production. All other routes limited to 100 per 15 minutes.
Nodemailer	6.10.1	Email sending for password reset flow. In development, automatically creates a free Ethereal test account — no SMTP configuration needed. In production, uses configured SMTP credentials.
csv-parse	5.6.0	Server-side CSV parsing for bulk student import. Every row is validated with Zod before insertion — malformed rows are reported back without aborting the whole import.
qrcode	1.5.4	Generates QR code data URLs for event check-in. Tokens are cryptographically random (32 bytes), SHA-256 hashed before storage, and expire 30 minutes after the event ends.

Testing

Technology	Why It Was Used
Jest + Supertest (backend)	Industry-standard Node.js testing. Supertest allows HTTP-level testing of Express routes without starting a real server. 44 tests covering auth, users, and events.
Vitest + React Testing Library (frontend)	Vitest is Vite-native — same config, near-instant test runs. React Testing Library encourages testing behaviour (what the user sees) rather than implementation details. 31 tests covering components, auth flow, and integration.

3. System Architecture

Directory Structure

```
campus-media-engagement-system/
├── client/ # React + Vite frontend
│ ├── src/
│ │ ├── api/ # Axios client + per-domain API modules
│ │ ├── components/
│ │ │ ├── ui/ # Design system: Button, Card, Input, DataTable…
│ │ │ ├── layout/ # AppShell, Sidebar, Navbar, ProtectedRoute
│ │ │ └── shared/ # Badges, modals, QR display, export button
│ │ ├── context/ # AuthContext (global auth state)
│ │ ├── hooks/ # useDebounce, usePagination, useNotifications
│ │ ├── pages/
│ │ │ ├── auth/ # Login, ForgotPassword, ResetPassword
│ │ │ ├── head/ # 15 Head Admin pages
│ │ │ ├── faculty/ # 8 Faculty pages
│ │ │ └── student/ # 8 Student pages
│ │ └── utils/ # formatters, validators
│ └── package.json
├── server/ # Node.js + Express backend
│ ├── controllers/ # Business logic (9 controllers)
│ ├── middleware/ # auth, authorize, validate, rateLimiter, errorHandler
│ ├── models/ # 7 Mongoose models
│ ├── routes/ # 9 Express routers
│ ├── schemas/ # 7 Zod validation schemas
│ ├── scripts/seed.js # Database seeder
│ ├── tests/ # Jest + Supertest tests
│ └── utils/ # audit, qr, mailer, csv helpers
└── package.json # Root scripts (dev, seed, test)
```

Request Flow

1. **Browser** — React component calls an API function (e.g. `listEvents()`)
2. **Axios client** — attaches httpOnly cookie automatically via `withCredentials: true`, sends request to `/api/events`

3. **Vite proxy (dev)** — forwards `/api/*` to `localhost:5000`. In production, Axios uses `VITE_API_URL` directly.
4. **Express middleware chain** — Helmet → CORS → rate limiter → cookie parser → mongo sanitize
5. **verifyToken middleware** — extracts JWT from cookie, verifies signature, attaches `req.user`
6. **authorize() middleware** — checks `req.user.role` against allowed roles. Returns 403 if not permitted.
7. **validate() middleware** — runs Zod schema against `req.body`. Returns 400 with field errors if invalid.
8. **Controller** — queries MongoDB via Mongoose, applies business logic, returns JSON response
9. **React component** — updates state with response data, re-renders UI

4. User Roles & Permissions

Every permission is enforced on the backend. UI elements are hidden for convenience only — a missing button is never the security boundary.

Head Administrator

- Manage blocks, faculty, students
- Create, publish, cancel events
- Assign supervisors & participants
- View all RSVPs and attendance
- Institution-wide analytics
- CSV bulk import / export
- Audit logs access
- Account activation / deactivation

Faculty Supervisor

- View assigned students only
- View supervised events only
- Mark attendance (manual / QR)
- Correct attendance with reason
- Finalize attendance register
- Send announcements to students
- Generate event QR codes
- View RSVP breakdown

Student

- View assigned events only
- Submit RSVP (Going / Not Going)
- Change RSVP before deadline
- QR check-in during window
- View own attendance history
- Receive notifications
- View own participation record
- Update own profile / password

5. Complete Event Workflow

1. **Block Setup** — Head creates academic blocks (ET, MT, FD, Pharmacy). Each block is assigned a faculty supervisor.
2. **User Registration** — Head registers faculty and students. Students are assigned a unique college-issued Student ID and linked to a block and supervisor.
3. **Event Creation** — Head or authorized faculty creates an event as a draft. Specifies date, time, venue, RSVP deadline, check-in window, participating blocks, assigned students, and supervisors.
4. **Publication** — Head publishes the event. EventParticipation records are auto-created for all assigned students with status *pending*. Notifications are sent.

5. **RSVP Collection** — Students log in, see the event on their dashboard, and submit Going or Not Going. They can change their response before the deadline.

6. **Faculty Review** — Faculty sees the live RSVP breakdown: how many going, not going, and pending. This is separate from attendance.

7. **Check-in Day** — Faculty opens the QR Attendance page, which generates a time-limited QR token. Students scan the QR (or manually enter the token) on their Check-in page. Each check-in creates an Attendance record. Duplicate check-ins are rejected by a unique compound index.

8. **Manual Recording** — Faculty also marks late arrivals, absences, and excused absences from the Attendance Register. All changes are logged in the correction history.

9. **Finalization** — Faculty finalizes the attendance register. Records become read-only. Further corrections require a written reason and create an audit trail entry.

10. **Head Review** — Head dashboard shows expected vs actual attendance, unresolved exceptions, and block-wise participation charts. Reports can be exported as CSV.

6. Database Models

User

- name, email, password (hashed)
- role: head | faculty | student
- studentId (unique, sparse)
- block → ref Block
- supervisor → ref User
- isActive, lastLoginAt
- passwordResetToken (hashed)
- passwordResetExpires

Block

- name, code (unique)
- description
- isActive
- createdBy → ref User

Event

- title, description, venue
- date, startTime, endTime
- status: draft|published|ongoing|completed|cancelled
- organizer, supervisors[], assignedStudents[]
- participatingBlocks[]
- rsvpDeadline, checkInWindow
- qrToken (hashed), qrTokenExpires
- cancelReason

EventParticipation

- event → ref Event
- student → ref User
- rsvpStatus: pending|going|not_going
- rsvpAt, changedAt
- **Unique index:** {event, student}

Attendance

- event → ref Event
- student → ref User
- faculty → ref User
- status: present|absent|late|excused
- checkInMethod: qr|manual|faculty_verified
- isFinalized
- correctionHistory[]

Notification

- recipient, sender → ref User
- type, title, message
- isRead
- relatedEvent → ref Event

- **Unique index:** {event, student}

AuditLog

- actor → ref User
- action (string)
- targetType, targetId
- changes: {before, after}
- ipAddress

7. Security Implementation

bcrypt (12 rounds) | JWT httpOnly cookies | Helmet headers | CORS allowlist | Rate limiting | Zod input validation | NoSQL injection prevention | Role-based authorization | Resource-level checks | Secure password reset tokens | QR token expiry | Duplicate check-in prevention | CSV row-level validation | Audit trail for corrections | Safe error responses

Key Security Decisions

- **httpOnly cookies** — JWT is stored in an httpOnly cookie, not localStorage. JavaScript cannot access it, eliminating XSS-based token theft.
- **Role never trusted from client** — The role is read from the verified JWT on the server. A student cannot escalate to head by modifying a request body.
- **Password reset token** — A 32-byte cryptographically random token is generated, SHA-256 hashed before storage, valid for 1 hour, and consumed on use. The raw token is only ever sent via email.
- **QR token hashing** — The raw QR token is sent to the student's browser but only the SHA-256 hash is stored in the database. The server compares hashes on check-in.
- **Unique compound indexes** — MongoDB enforces one RSVP per student per event and one attendance record per student per event at the database level — not just in application code.

8. Dashboards & Pages

Head Administrator — 15 Pages

Page	What it shows
Overview / Dashboard	Stats (students, faculty, blocks), Recharts participation chart, recent events, attendance exceptions
Events	Paginated event list with status filters, search, create button
Create / Edit Event	Full event form with block selector, student multi-select, supervisor assignment
Event Detail	RSVP breakdown, participant list, publish / cancel actions
Blocks	Create, edit, deactivate academic blocks
Students	Paginated directory, search by name/ID, filter by block, activate/deactivate
Student Detail	Profile, participation history, attendance history
CSV Import	Upload CSV, see row-level results (success/errors), download template
Faculty	Faculty list, add faculty, activate/deactivate
Faculty Detail	Assigned students, supervised events
Attendance Overview	Filter by event/block/date, expected vs actual table
Reports	Attendance & participation reports with CSV export
Notifications	All notifications, mark read/unread
Audit Logs	Paginated log of all sensitive admin actions
Settings	Change password, update profile

Faculty Supervisor — 8 Pages

Page	What it shows
Dashboard	Assigned students count, upcoming events, RSVP summary, exceptions
My Events	Events I supervise with status and participant counts
Event Detail	Participant list with RSVP status, links to attendance and QR
Attendance Register	Full register: RSVP vs attendance per student, set status, correct with reason, finalize
QR Attendance	Live QR code display for student check-in, refresh button
My Students	All students under my supervision with attendance summary
Announcements	Send announcement to all my assigned students
Notifications	My notifications with read/unread state

Student — 8 Pages

Page	What it shows
Dashboard	Welcome card (name, ID, block, supervisor), upcoming events with RSVP buttons, recent notifications
My Events	All assigned events with RSVP status, filter by upcoming/past
Event Detail	Full event info, RSVP action, check-in button when window is open
QR Check-in	Token entry form, success/already-checked-in/window-closed states
My Participation	All my RSVPs with event details and attendance outcome
Attendance History	My full attendance record filtered by status
Notifications	All my notifications
Profile	View profile, change password

9. Local Development Setup

Prerequisites

- Node.js 18 or higher
- MongoDB (local) or MongoDB Atlas URI
- npm 9 or higher

Commands

```
# Step 1: Navigate to project root
cd campus-media-engagement-system

# Step 2: Install all dependencies (run once)
npm run install:all

# Step 3: Seed the database (run once, or to reset)
npm run seed

# Step 4: Start both servers
npm run dev

# Frontend → http://localhost:5173
# Backend → http://localhost:5000

# Run all tests
npm test
```

Default Login Credentials

Role	Email	Password
Head Administrator	head@cmes.edu	Head@2024!
Faculty – ET	faculty.et@cmes.edu	Faculty@2024!

Role	Email	Password
Faculty – MT	faculty.mt@cmes.edu	Faculty@2024!
Faculty – FD	faculty.fd@cmes.edu	Faculty@2024!
Faculty – Pharmacy	faculty.pharmacy@cmes.edu	Faculty@2024!
Student – ET Block	ET2024001@student.cmes.edu	Student@2024!
Student – FD Block	FD2024001@student.cmes.edu	Student@2024!

Environment Variables (server/.env)

```
MONGO_URI=mongodb://localhost:27017/cmes
JWT_SECRET=your_random_secret_here
JWT_EXPIRES_IN=24h
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:5173
```

10. API Reference

Method	Endpoint	Access	Description
POST	/api/auth/login	Public	Authenticate, set JWT cookie
GET	/api/auth/me	All	Get current user profile
POST	/api/auth/logout	All	Clear JWT cookie
POST	/api/auth/forgot-password	Public	Send password reset email
POST	/api/auth/reset-password/:token	Public	Reset password with token
GET	/api/users	Head	List users (paginated, filterable)
POST	/api/users	Head	Create faculty or student

Method	Endpoint	Access	Description
POST	/api/users/bulk-import	Head	CSV bulk import students
GET	/api/blocks	All	List all blocks
POST	/api/blocks	Head	Create block
GET	/api/events	All	List events (role-filtered)
POST	/api/events	Head/Faculty	Create event
PATCH	/api/events/:id/publish	Head	Publish draft event
PATCH	/api/events/:id/cancel	Head	Cancel event
GET	/api/events/:id/qr	Faculty/Head	Generate QR code data URL
POST	/api/participation/:eventId/rsvp	Student	Submit RSVP
GET	/api/participation/my	Student	My participation records
GET	/api/attendance/:eventId	Faculty/Head	List attendance for event
POST	/api/attendance/qr-checkin	Student	Check in via QR token
POST	/api/attendance/:eventId/manual	Faculty/Head	Manual attendance entry
PATCH	/api/attendance/:eventId/finalize	Faculty/Head	Finalize attendance register
GET	/api/reports/attendance	Head	Attendance report
GET	/api/reports/export/attendance	Head	Download attendance CSV
GET	/api/notifications	All	My notifications
POST	/api/notifications/announce	Faculty	Announce to my students
GET	/api/audit-logs	Head	Paginated audit log

Campus Media Engagement System — Technical Documentation v1.0

Built with MERN Stack · React 18 · Express 5 · MongoDB · Tailwind CSS